

Increasing True Risk by Banning Jump and Other Corruption Gameplay Verbs

Why Removing a Verb Can Make Every Other System Stronger

Registry: [HOWL-GAME-2-2026]

DOI: 10.5281/zenodo.22964296

Date: September 2026

Domain: Applied Philosophy

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Opus 4.8.

Part 1 — Purpose and audience

This paper is for a game designer who has not encountered this argument before. It explains why certain player verbs, jumping most of all, weaken the games they appear in, and why removing them makes the remaining systems stronger. It builds the argument one step at a time. Each term is defined before it is used. There is no assumed background beyond having played a few games.

The claim of the paper is narrow and precise. It is not that jumping is bad. It is that jumping, and a small set of similar verbs, are destructive when they are not the central mechanic of the game. The paper defines “central mechanic,” defines the harm precisely, shows the mechanism of the harm, and then shows the repair.

Part 2 — The two representations of a game world

Every game with movement holds its world in two separate representations at the same time.

The first is the **collision representation**. This is the physics layer. It knows about solid volumes, gravity, and what stops a body from passing through what. It is applied almost everywhere in the world because it is cheaper to apply collision everywhere than to carve out exceptions.

The second is the **navigation representation**. This is the graph of where an actor is *intended* to be able to go. It is usually a navmesh (a mesh of walkable surfaces), a waypoint graph, or a tile grid. Its edges are the legal moves: cell A connects to cell B if an actor may walk from A to B. This graph is what the game's systems reason about when they ask any question involving position.

The important fact is that these two representations are not required to agree. The collision layer may permit a body to occupy a spot that the navigation graph never connected to anything. Most of the time this gap does not matter, because the player, like every non-player actor, moves by walking along navigation edges.

Jumping is the verb that lets the player leave the navigation graph and operate directly on the collision layer.

Part 3 — What “reasoning about position” means

Many game systems ask questions about where an actor is. A short list:

- Is this region sealed, so nothing can get in or out?
- Can a pursuer reach the player to attack?
- Is this building exposed to weather?
- Can this actor flee to safety?
- Is the player inside the guarded area or outside it?

Every one of these questions is answered using the navigation graph, because the navigation graph is the definition of where actors can be. A “sealed” region is one the graph does not connect to the outside. A “reachable” target is one the graph connects a path to.

This gives a single guarantee that the whole game depends on: **the set of places the player can be is the same as the set of places the navigation graph says the player can be.** While this guarantee holds, every position-based system is correct, because they all read from the one graph and the player lives inside that one graph.

Part 4 — How jumping breaks the guarantee

Jumping queries the collision layer, not the navigation graph. A jump can carry the player to a spot the graph never connected to anything. The moment that happens, the guarantee in Part 3 is broken. The player is now in a place the systems believe is unreachable.

Every position-based system is now reasoning about a world the player has left:

- The “sealed” check still reports sealed, but the player is inside. Containment, quarantine, and prison silently fail.
- The pursuer’s path to the player is invalid, because the player is off the graph. The enemy cannot reach a spot the graph does not contain, so combat degrades.
- The “exposed to weather” check is wrong, because being in a hazard region is defined by graph membership, and the player is not a member.

- The fleeing actor's escape route is computed on a graph the player has abandoned, so its behavior is computed against a false world.

This is the core harm. Jumping does not add one traversal option. It voids the shared assumption that position implies graph membership. Once that assumption is void, no position-based rule can be trusted, because any of them can be presented with a player state it has no rule for.

Two well-known examples make this concrete. In one large fantasy game, a player could jump onto ledges the navigation graph did not connect to, sit in a combat arena at a spot no enemy could path to, and defeat every opponent with ranged attacks unopposed. In another, a player could chain jumps up mountain faces that were deliberately left off the navigation graph as impassable, crossing terrain the systems treated as a wall. In both cases the player did not defeat the traversal system. The player left it.

Part 5 — Why leaving the systems is worse than it looks

A single-player game whose depth comes from its systems is a stack of systems that act on each other. Consider a chain of consequences:

1. You must cross a river, because there is no way around it.
2. Crossing the river writes a state onto you: you are wet.
3. Being wet in cold air writes another state: you are losing body heat.
4. Being cold applies a penalty when you next fight.
5. Fighting while penalized changes the outcome of the encounter.

Each link exists only because the previous link was unavoidable. The wet state means something only because you could not skip the river. The cold means something only because you were wet. The combat penalty means something only because you were cold.

Jumping cuts the chain at the first link. If you can jump the river, you are never wet, so you are never cold, so there is no combat penalty, so the encounter is just a fight. Every downstream system that depended on the first system firing is now dead, not because those systems are shallow, but because their precondition was bypassed.

This produces the general result: **a system whose precondition can be bypassed is a system that never reliably fires, and a system that never reliably fires is not worth making deep.** So a game with a universal bypass verb like jumping cannot afford deep coupled systems, because the player who would most benefit from the depth is exactly the player routing around it. The shallowness of such games is not a failure of imagination. It is a direct consequence of the bypass verb.

Removing the bypass reverses this. When the only traversal is walking along the navigation graph, the player arrives at every system's precondition legitimately. The river cannot be skipped, so wet fires, so cold fires, so the penalty fires, so the fight inherits the whole

journey. Removing the verb does not take a feature from the player. It grants authority to every system that reasons about position and state, because now those systems can trust their inputs and are therefore worth deepening.

Part 6 — Central mechanic versus side mechanic

The argument so far seems to condemn jumping outright. It does not. The distinction is whether jumping is the central mechanic or a side mechanic.

A **central mechanic** is the verb the game is built around. Its content exists to be engaged by that verb.

A **side mechanic** is a verb the game offers in addition to its central mechanics.

In a platform game built on jumping, jumping is central. Every platform, gap, enemy, and timing challenge exists to be engaged by the arc. Jumping does not bypass the game's systems, because jumping is the game's system. A verb cannot steal from the depth when it is the depth.

In a role-playing game, the central mechanics are typically stats, items, and the map. Jumping there is a side mechanic. And here is the exact test for harm: a side mechanic is destructive when it can change the value of the central mechanics from outside them.

Jumping over a mountain range changes the value of the map, because geography stops constraining the player. Jumping into an unreachable position changes the value of stats and items, because position wins fights the player's build should lose. A side verb that alters the worth of the central mechanics is not a feature. It is a lever on the game's own currency, operated from outside the game's rules.

So the test is a value test. For any verb: does it change the value of the central mechanics from outside them? If yes, it is a side mechanic defeating a central one. It must be made central, made costly (Part 8), or removed. The same verb, jumping, is legitimate in one game and corrosive in another, entirely according to whether it is the axis the game turns on or a bypass around that axis.

Part 7 — Two kinds of game, and what a bypass steals from each

To see what a bypass verb actually steals, separate games into two kinds by where their depth comes from.

Full-knowledge games with an opponent. Chess and Go are examples. There is no hidden information. Their depth cannot come from what the player does not know, because the player knows everything. It comes from another mind computing against the player over a space too large to exhaust. The opponent is the source of difficulty. A very small such game, like tic-tac-toe, can be fully solved and so produces the same experience every time. A large one produces unique games. In this kind of game, a bypass verb that

skips a wall steals little, because the challenge was never the wall. It was the opponent, and the opponent is still there.

Hidden-knowledge single-player games. A large open-world role-playing game or a single-player colony simulation are examples. There is no opponent mind. Their depth cannot come from an adversary. It comes from the experience of moving through a world whose state the player does not fully know, under systems that commit their consequences. The source of value is the world plus the player's own committed choices.

This second kind is exactly where bypass verbs are lethal. The game's entire value is the experience of being subject to the systems. Any verb that lets the player exit a system deletes a piece of the only thing the game was offering, and there is no opponent to keep the game hard once the systems are left. This is the precise meaning of "cheating the experience": the game had one thing to give, the lived consequence of its systems, and the bypass took a piece of it and returned nothing the game valued.

Part 8 — True Risk, and why it is the repair

The repair for a corrupting verb is not always removal. Sometimes the verb can be kept if it is given **True Risk**.

True Risk is defined as follows: a decision carries True Risk when its outcome set contains a bad branch that is reachable, that commits when the decision is made, and that can only be undone by playing forward. The player does not always pay the bad outcome, but they might, and the possibility is real and permanent at the moment of choosing.

A verb with True Risk is a genuine wager against the world. A verb without it is a formality that always resolves in the player's favor, given enough attempts.

The clearest example is a lockpick with a single chance. You may attempt the lock. If you fail, the lock is damaged so that even the correct key no longer opens it, and now you must break through, which is loud and costly, or leave it. The bad branch is reachable, it commits on failure, and it is permanent this run. That commitment is what makes the attempt a real choice rather than a delay.

Compare a lockpick with unlimited retries. The bad branch never commits, because failure is only a pause before the next attempt. Such a lockpick has no True Risk. It is a formality wearing the costume of a challenge.

There is a prerequisite for True Risk that must be stated separately, because one common verb destroys it universally.

Part 9 — Save-scumming, the universal destroyer of True Risk

Saving and reloading at will, often called save-scumming, lets the player choose when the world's state commits. Whenever an outcome is unfavorable, the player reloads and tries

again.

This does not steal one system. It reaches into every mechanic that has a committed bad branch and un-commits it, converting every wager back into a formality. A one-chance lockpick under free reloading is exactly an unlimited-retry lockpick, because the reload restores the pre-failure state. A permanent injury under free reloading is not permanent. A lost gamble is not lost.

Therefore True Risk has a prerequisite: **the world's state must commit at the moment of the wager and stay committed.** Any mechanic that lets the player control when state commits, save-scumming being the primary one, voids the prerequisite for every risk-bearing system at once. In a game whose depth is committed consequence, this verb must be constrained first, before any other, through autosave-only, single-slot, or ironman rules. Otherwise no other consequence in the game is load-bearing.

Part 10 — The lockpicking case, and the Thief exception

Lockpicking as commonly implemented is a corrupting verb of the same class as jumping. It is a free, instant, general solution to closed containers. It voids the specific, costed answers the game had for getting what is behind a lock:

- Find the key, which is a search with its own cost.
- Buy the key, which spends money that had other uses.
- Bribe someone, which spends money and involves a person.
- Pick a pocket for the key, which is a risky action with a bad branch.
- Break the container, which makes noise and draws attention.

Each of these is more gameplay than a lockpick, because each is a costed action embedded in another live system: economy, social interaction, stealth, or noise-and-detection. A free instant lockpick collapses that whole fan of options into a single skill check with no True Risk, and every option it replaces was worth more than the verb that replaced them.

The exception proves the rule. In the game *Thief*, lockpicking is legitimate, because it is not free and not instant, and it is embedded in a live system. Picking a lock takes real time, and during that time guards patrol on their routes. The pick is therefore not a bypass of the challenge. It is a way of spending time inside the challenge, and the time spent is exposure to the stealth system, which can catch you mid-pick. The lock is redeemed because the verb was made costed and threatened. It stopped being an exit from the systems and became an action the systems get to act upon.

This gives the general repair for any corrupting verb: take away its freeness and its instantness, embed it inside another live system that prices and threatens it, and where the verb involves position, validate its outcomes so they land back on the navigation graph. A costed, threatened, on-graph version of a verb is a move within the game. The free, instant, unconstrained version is an exit from it.

The one-chance-damages-the-lock system from Part 8 is a second valid repair, by a different route: it adds True Risk directly rather than embedding the verb in a stealth system. Either repair works because either one gives the verb a committed bad branch.

Part 11 — The general enumeration

Jumping and lockpicking are two instances of one pattern. A mechanic steals more than it provides when it is a free, instant, general solution to a problem the game had specific, costed, systemic answers to. The theft is measured by how many distinct systems the general solution makes optional, because every system a bypass reaches is a system not worth deepening. The following verbs are the same pattern, each with the systems it voids and the repair that redeems it.

Fast travel is a general solution to distance. It voids the mount, the learned road, the ambush on the route, the terrain constraint, the supplies the journey spends, and the encounters that make the world feel inhabited. Repair: make it costed and interruptible, so travel spends time and supplies and can be attacked, and is a decision rather than a skip.

Charm or persuasion as a universal dialogue solver is a general solution to social obstacles. It voids the earned trust, the completed quest, the payment, the real leverage, and the threat backed by actual force. Repair: gate persuasion on real prior state, so the check reads a genuine relationship record rather than a floating skill number.

Invisibility as a toggle is a general solution to detection. It voids line of sight, patrol routes, sound, light, and guard schedules, because the player is no longer in the detection graph. Repair: make it costed and leaky, draining a resource, breaking on action, still making sound, so it is a tool used inside the stealth system rather than an exit from it.

Infinite summoned allies is a general solution to being outmatched. It voids party composition, recruitment cost, the irreplaceability of real units, and positioning. Repair: make summons costed and finite, with real upkeep and permanent loss.

Infinite carry weight is a general solution to logistics. It voids hauling, trip cost, the decision of what to bring, and storage choices. Repair: do not grant it, because weight and trips are the constraint; let a technology ladder reduce the cost rather than a switch that removes it.

Detect-everything or full-reveal information is a general solution to not knowing. It voids exploration, scouting, learned routes, the ambush behind the blind corner, and the informant. Repair: make detection partial, costed, and directional, so information is spent to get rather than simply had.

Free resurrection is a general solution to death. It voids permanent loss, the weight of a lost member, and the stakes that make combat catastrophic. Repair: make revival rare, heavily costed, and consequential.

Crafting anything from generic materials is a general solution to acquisition. It voids the market, the trader, scarcity, and the trade decisions those create. Repair: gate crafting

on specific scarce inputs, real time, and a station that competes for labor, so it is one costed source among several.

Mind control or dominate is a general solution to both enemies and labor. It voids combat as a contest and recruitment as a cost, stealing from two economies at once. Repair: make it costed, temporary, and risky.

Blink or short-range teleport is a general solution to local obstacles, the tactical-scale form of jumping. It voids going around, finding the gap, waiting for the patrol, and opening the door. Repair: validate the destination so the player can only warp where they could already reach, and cost the verb.

Instant or automatic healing is a general solution to attrition. It voids the injury model, the recovery time, and the rest-food-warmth chain that is meant to be the thing that heals. Repair: make healing slow, resource-costed, and incomplete, so injury is a persisted state other systems can act on.

The unifying rule is one statement. A verb belongs in a game only if it satisfies two conditions. First, it must carry True Risk: a reachable, committed, play-forward-only bad branch, which no verb has under save-scumming, so save-scumming is disqualified first in any consequence game. Second, it must be either the central mechanic, which cannot steal from itself, or a side mechanic that joins the central ones as a costed peer without changing their value from outside. A verb that is free, instant, riskless, and able to alter the game's own currency from outside its rules is a cheat code by definition, and players will find it, because it sits at the attack surface of every wall, lock, and reload in the world.

Part 12 — Applying the rule to a design

The rule gives a procedure for evaluating any proposed verb.

First, name the class of problem the verb generally solves.

Second, list the specific systems that were the good answers to that class of problem.

Third, decide whether the verb makes those systems optional or joins them as one more costed option.

If it makes them optional, the verb is a side mechanic defeating central mechanics. It must be made central, so the game is built around it and its content exists to be engaged by it, or it must be given True Risk and embedded in a live system that prices and threatens it, with any positional outcome validated back onto the navigation graph, or it must be removed.

Two positive examples show the constrained form.

A **fall** verb can be given to the player that the non-player actors do not have, allowing the player to drop from a higher tile to a lower one, but only when the tile directly below is already walkable. Its outcome is bounded: it saves a little time, and it risks injury or death.

It can never place the player off the navigation graph, because the landing tile was already a graph member. It is an exclusive player advantage that is nonetheless fully inside the ruleset, so it cheats no system.

A **tactical jump** can restore ballistic movement without reopening the exploit. The player places a target with an arc, and the landing is validated before the jump is legal, including a check that the landing tile connects back to the walkable world so the player cannot strand themselves or, by the same graph, the non-player actors. A jump that can only end on a validated, connected tile is a jump that cannot produce an off-graph state, so it cannot disable any position-based system. It also carries True Risk: the attempt can fail, and a failed attempt can drop the player into a modeled bad state, such as falling backward into water, which is itself a real system with its own consequences. This turns the one corrupting verb into a deterministic edge the whole system already understands.

Part 13 — Summary

A game world is held in two representations, collision and navigation, that are not required to agree. Every system that reasons about position reads the navigation graph. The player's value to those systems depends on a single guarantee: that the player can only be where the graph says the player can be.

Jumping breaks that guarantee by moving the player onto the collision layer and off the graph, which silently disables every position-based system by presenting it with a state it has no rule for. This is worse than losing one verb's worth of challenge, because the systems of a deep single-player game are chained, and a bypass at the first link drops every system downstream, which is why games with a universal bypass verb cannot afford deep coupled systems.

Removing the verb does not subtract a feature. It grants authority to every system that reasons about position and state, because those systems can now trust their inputs and are worth deepening.

A verb is legitimate only if it is the central mechanic the game is built around, or a costed side mechanic that carries True Risk and does not change the value of the central mechanics from outside them. True Risk requires a committed bad branch, which save-scumming destroys universally and must therefore be constrained first. Lockpicking, fast travel, invisibility, summoning, infinite inventory, full-reveal information, resurrection, universal crafting, mind control, teleport, and instant healing are all the same pattern as jumping, and each is redeemed by the same operation: remove its freeness and instantness, give it a committed bad branch, embed it in a live system that prices and threatens it, and validate any positional outcome back onto the graph.

The test for a designer is a single question asked of every verb. Does this let the player change the value of the central mechanics without going through them? If it does, it is a corruption verb, and it must be made central, made costed with True Risk, or removed.

HOWL-GAME-2: Supporting Appendix

These appendices carry material the paper referenced but did not tabulate, plus connected material that did not appear in the body: the exact classification tests, the two-representation failure map, the coupling-chain formalism, the full corruption-verb ledger with its redemption column, the multiplayer boundary case, the exclusive-verb design space, and the decision procedure stated as a table. No content from Parts 1–13 is repeated; these connect and complete it.

Appendix A — The Two-Representation Failure Map

The body stated that collision and navigation are separate representations that need not agree, and that jumping moves the player from the second onto the first. This table names, for each position-reasoning system, the exact query it runs, what that query reads, and the specific false answer an off-graph player produces.

System	Query it runs	Reads	False answer when player is off-graph
Sealing / containment	is interior disconnected from exterior	navigation graph connectivity	reports sealed; player is inside
Pursuit	is there a path from pursuer to player	navigation graph path search	no path found; enemy cannot engage a valid target
Weather / hazard exposure	is this tile in the hazard region	graph membership of the tile	player in hazard reads as not-in-hazard
Flee / escape routing	shortest path to safety	navigation graph	AI routes on a world the player has left
Guard zone / detection geometry	is player inside the watched area	graph cell membership	player present but not counted as present
Reachability of objectives	can the player legally get here yet	graph connectivity from start	objective reached before its gate opened
Quest barrier	is the far side still blocked	graph edge absence	barrier bypassed, sequence broken
Territory control (multiplayer)	which tiles does a player hold	occupation over the graph	control asserted from a cell not in the contest

The single row that generalizes all of them: every system in this table defines its predicate over graph membership, so a player who is present-in-collision but absent-in-graph makes every predicate return the wrong value simultaneously. This is why the harm is not additive across systems but is one shared failure surfacing in many places.

Appendix B — The Coupling-Chain Formalism

The body used a river-to-combat chain as an example. This appendix states the chain form generally, because the strength of a deep game is measured in chain length, and the harm of a bypass verb is measured in how early it cuts.

A **coupling chain** is a sequence of systems $S1 \rightarrow S2 \rightarrow \dots \rightarrow S_n$ where each S_i writes a state that is a precondition of S_{i+1} . The chain has these properties:

Property	Statement	Consequence
Precondition dependence	S_{i+1} fires only if S_i fired	a cut at S_i drops $S_{i+1} \dots S_n$
Cut cost	cutting the chain requires reaching S_i 's precondition by a route S_i does not model	this is exactly what a bypass verb provides
Depth value	the game's felt depth is proportional to reliable chain length	shallow games have short or unreliable chains
Deepening incentive	a system is worth deepening only if its precondition reliably fires	a bypassable S_i makes $S_{i+1} \dots S_n$ not worth deepening

The design rule that falls out of the table: **you cannot deepen S_{i+1} profitably while S_i is bypassable**. This is the formal reason a game with a universal bypass verb converges to shallow systems regardless of designer effort. The effort spent on S_n is wasted because the player who most wants S_n 's depth is the one who cut S_1 . Removing the bypass is therefore not a balance change; it is the precondition for the deepening work to have any return.

A worked chain from the paper's own domain, to show the form is not toy-sized:

river-crossing $\rightarrow$ wet $\rightarrow$ cold $\rightarrow$ combat penalty $\rightarrow$ lost fight $\rightarrow$ lost body $\rightarrow$ lost labor $\rightarrow$ water unhailed $\rightarrow$ cattle die $\rightarrow$ less money $\rightarrow$ slower recruitment $\rightarrow$ fewer hands.

Twelve links. A jump verb cuts it at link 1. Every one of the eleven downstream systems becomes optional for the jumping player, which is eleven systems not worth deepening, which is the entire simulation's depth, defeated by one side verb.

Appendix C — The Classification Tests, Stated Exactly

The body gave three tests informally (the value test, the central/side test, the two-game-type test). This table states each as a yes/no procedure a designer can apply mechanically.

Test	Question	If yes	If no
Value test	does the verb change the value of a central mechanic from outside it	it is a corruption verb; go to repair	it is safe as a peer option
Central/side test	is the game built around this verb, with content existing to be engaged by it	it is central; it cannot steal from itself	it is a side mechanic; apply the value test
Game-type test	does the game's depth come from an opponent mind	bypass steals little; the opponent remains	bypass steals the experience; guard strictly
True Risk test	does the verb have a reachable bad branch that commits at choice time	it is a wager; keep it	it is a formality; add risk or remove
Commit test	can the player choose when world state commits	True Risk is void everywhere; fix this first	risk-bearing verbs are viable

The tests are ordered by priority. The Commit test dominates all others: if it fails, no other verb's True Risk means anything, so it is resolved before evaluating any individual verb. This is the formal placement of save-scumming as the first thing constrained in any consequence game.

Appendix D — The Full Corruption-Verb Ledger

The body enumerated the verbs in prose. This table gives the complete ledger with a uniform four-column shape: the problem class the verb generally solves, the specific systems it voids, whether it can be central, and the redemption operation. The uniform shape is the point — every row is the same pattern, which is the paper's thesis in tabular form.

Verb	General problem it solves	Specific systems it voids	Can be central?	Redemption
Jump	traversal past an obstacle	reachability, sealing, pursuit, exposure, slope, curfew, fall-damage, stealth geometry	yes (platformer)	validate landing on-graph + True Risk (tactical jump)
Lockpick	opening a closed container	find/buy/bribe/pickpocket/smash, each its own system	yes (lock-game)	cost it in time under a live threat (Thief); or one-chance-damages-lock
Fast travel	crossing distance	mount, road knowledge, route ambush, supplies, encounter density	rarely	costed, interruptible, raidable travel
Persuasion (universal)	social obstacle	trust, quests, payment, leverage, force-backed threat	yes (a talk-game)	gate on real relationship state, not a floating skill
Invisibility (toggle)	detection	sight, sound, light, patrols, schedules	yes (a stealth-game)	costed, leaky, breaks on action
Infinite summons	being outmatched	composition, recruitment cost, unit irreplaceability, positioning	rarely	finite, costed, permanently losable summons
Infinite inventory	logistics	hauling, trip cost, what-to-bring, storage choice	no	do not grant; tech ladder reduces, never removes
Full-reveal info	not knowing	exploration, scouting, learned routes, ambush, informants	rarely	partial, costed, directional detection

Verb	General problem it solves	Specific systems it voids	Can be central?	Redemption
Resurrection (free)	death	permanent loss, member weight, catastrophic stakes	rarely	rare, heavily costed, consequential revival
Universal crafting	acquisition	market, trader, scarcity, trade decisions	yes (a crafting-game)	gate on scarce inputs, time, competing labor
Mind control	enemies + labor	combat as contest, recruitment as cost	rarely	costed, temporary, risky
Blink / teleport	local obstacle	go-around, gap-finding, patrol-waiting, doors	yes (a movement-game)	validate destination reachable + cost
Instant healing	attrition	injury model, recovery time, rest-food-warmth chain	no	slow, costed, incomplete healing
Save-scum	consequence itself	every committed-branch system at once	never	remove the commit choice: ironman, single-slot, autosave-only

The two rows that differ in kind: **Save-scum** is the only verb whose “systems voided” column is “all of them,” which is why its redemption is not modification but removal of the capability, and why it is handled before every other row. **Infinite inventory** and **Instant healing** are the two rows marked “no” under central, because there is no coherent game whose central axis is carrying-unlimited-weight or taking-no-lasting-damage; these are always side mechanics and always corruptions, with reduction-not-removal as their only path.

Appendix E — The Redemption Operations, Decomposed

The body said every corrupting verb is redeemed by “the same operation.” That operation has four independent components, and a given verb may need one, some, or all. This table

separates them so a designer knows which components a specific verb requires.

Component	What it does	Which verbs need it
De-free	attach a resource cost to the verb	all
De-instant	make the verb take time during which systems act	verbs whose harm is speed of bypass (lockpick, fast travel, persuasion)
On-graph validation	require positional outcomes to land on the navigation graph	positional verbs (jump, blink, fall)
True Risk	add a reachable committed bad branch	all, but especially verbs the player would otherwise spam

The independence matters. A tactical jump needs on-graph validation and True Risk but is already discrete enough not to need de-instant. Thief's lockpick needs de-free and de-instant but not on-graph validation, because a lock is not a position exploit. Reading the table this way prevents over-correction: you apply only the components the specific harm requires, rather than burdening every verb with all four.

Appendix F — The Exclusive-Verb Design Space

The body introduced two positive constructions, fall and tactical jump. This appendix places them in a small design space, because the interesting question is not only “what to ban” but “what exclusive verb can a player safely have that non-player actors do not.” The axis is: how far outside the shared graph does the verb's outcome set reach.

Verb	Outcome set relative to graph	Exclusive to player?	True Risk present?	Verdict
Walk	entirely on-graph, all edges shared	no	no	the baseline; safe by definition
Fall	on-graph; lands only on already-walkable tile below	yes	yes (injury/death)	safe exclusive advantage; saves time, risks harm

Verb	Outcome set relative to graph	Exclusive to player?	True Risk present?	Verdict
Tactical jump (validated)	on-graph; landing validated + connectivity checked	optional	yes (failure drops to modeled bad state)	safe; a deterministic added edge
Tactical jump (unvalidated)	may leave graph	optional	partial	unsafe; reintroduces the exploit corruption verb; the paper’s subject
Ordinary jump	routinely off-graph	usually	no	

The rule the table encodes: an exclusive player verb is safe exactly when its entire outcome set, including its failure outcomes, stays inside the shared graph. Fall is the minimal such verb — one extra capability, strictly bounded, strictly on-graph, strictly risk-bearing. It demonstrates that “the player may have something the NPC lacks” is not the danger; “the player may reach a state the systems do not model” is the danger, and those are different claims. A verb can grant the first without granting the second.

Appendix G — The Multiplayer Boundary Case

The body’s two-game-type split said full-knowledge games with an opponent are relatively safe from bypass verbs because the opponent remains. This appendix states the one condition under which that safety fails, because it is not automatic.

In a full-knowledge game with an opponent, a bypass verb is safe **only if the verb’s outcome stays inside the state space both players are reasoning about**. The opponent is the entropy source, but the opponent computes over the same shared representation. If a bypass verb lets one player reach a state the opponent’s reasoning does not cover, the verb corrupts the contest even though an opponent exists.

Condition	Bypass verb effect	Example
Verb stays in shared state space	safe; opponent adapts	a legal chess move the opponent did not expect
Verb exits shared state space	corrupts; opponent cannot reason about it	a positional exploit that puts a unit where the game’s own rules cannot place it

This is why a competitive colony game built on the same no-jump substrate stays honest: both players and all non-player actors share one navigation graph, so any state one player can reach, the other's reasoning already covers. The opponent's presence is a sufficient defense against bypass only when the bypass cannot leave the representation the opponent shares. Remove jumping and that condition holds automatically; keep jumping and even the presence of a human opponent does not save the contest, because the exploit is outside the space the opponent is playing in.

Appendix H — The Decision Procedure as a Table

The body's Part 12 gave the procedure in prose. This is the same procedure as a lookup, for use during design review of any proposed verb.

Step	Ask	Outcome A	Outcome B
1	Can the player choose when world state commits?	fix commit first (Appendix C); then continue	continue
2	What class of problem does this verb generally solve?	name it	—
3	What specific systems were the good answers to that class?	list them	—
4	Does the verb make those systems optional?	it is a corruption verb; go to 5	it is a safe peer option; accept
5	Can the game be built around this verb as central?	make it central; accept	go to 6
6	Apply the redemption components (Appendix E) that the harm requires	verb redeemed; accept	if it cannot be redeemed, remove

The procedure terminates in one of three states for every verb: accepted as a safe peer, accepted as central, accepted as redeemed, or removed. There is no fourth terminal state, which restates the body's claim that a free, instant, riskless verb able to alter the game's currency has no legitimate form except one of these.

Appendix I — Terminology, Consolidated

The paper introduced terms across many parts. This table collects them with one-line definitions for reference.

Term	Definition
Collision representation	the physics layer of solid volumes and gravity, applied nearly everywhere
Navigation representation	the graph of intended legal moves, read by all position-reasoning systems
The guarantee	the player can only be where the navigation graph says the player can be
Off-graph state	a position permitted by collision but not connected in navigation
Position-reasoning system	any system whose predicate is defined over graph membership
Coupling chain	a sequence of systems where each writes a precondition of the next
Central mechanic	the verb the game is built around; content exists to be engaged by it
Side mechanic	a verb offered in addition to the central mechanics
Value test	does the verb change a central mechanic's value from outside it
True Risk	a decision with a reachable bad branch that commits at choice time and is undoable only by playing forward
Commit prerequisite	world state must commit at the wager and stay committed
Corruption verb	a free, instant, general solution that makes specific costed systems optional
Redemption	de-free, de-instant, on-graph validation, and True Risk, applied as the harm requires
Exclusive verb	a capability the player has that non-player actors lack

The consolidated list makes visible that the entire paper is built from a small vocabulary, and that every argument in it is a statement about how these dozen terms relate. This is the same economy the paper recommends for game verbs applied to its own concepts: a few load-bearing definitions, each doing work everywhere, none of them a bypass around the others.

[[HOWL-GAME-2-2026](https://github.com/ghowland/papers/tree/main/papers/GAME/HOWL-)] Increasing True Risk by Banning Jump and Other Corruption Gameplay Verbs. Github: <https://github.com/ghowland/papers/tree/main/papers/GAME/HOWL->

GAME-2-2026